

Modelos Bioinspirados y
Heurísticas de Búsqueda
4 Curso Grado Ingeniería en
Informática

Área de Ciencias de la Computación e
Inteligencia Artificial
Departamento de
Tecnologías de la Información

AAD (Versión 2026, 1.0)

Algoritmos de Optimización Basados en Colonias de Hormigas

Objetivos

Objetivos

El objetivo de esta actividad dirigida es estudiar el funcionamiento de los *Algoritmos de Optimización Basados en Colonias de Hormigas (OCH)*. Para ello, se requerirá que el alumno implemente distintas variantes de estos algoritmos, correspondientes al *Sistema de Hormigas (SH)*, *Sistema de Hormigas Elitista (SHE)* y *SCH* (Sistema de colonias de hormigas). El comportamiento de los algoritmos de OCH implementados deberá compararse con un *algoritmo greedy*.

Para ello, se requerirá que el alumno implemente estos algoritmos, el para resolver el problema de optimización de balanceo de una red de bicicletas en una ciudad visto en las prácticas 1, 2 y 3 eliminando el problema de capacidad del camión, por lo que sólo deberá recorrerse el conjunto de estaciones en la menor distancia posible. El comportamiento de los algoritmos implementados deberá compararse entre sí y con un Greedy aleatorizado de lista corta de tamaño 3. El análisis de los resultados influirá decisivamente en la calificación de la práctica

Enunciado de la práctica

Representación y Evaluación

La representación de tipo matriz para las unidades y localizaciones, así como el método de evaluación, serán las empleadas en las prácticas anteriores.

El alumno implementará el SH (Sistema de Hormigas) SHE (Sistema de Hormigas Elitista) y SCH (Sistema de colonias de hormigas) para resolver el problema, con los distintos *data sets*.

A continuación, mostramos los valores de los parámetros a considerar en las ejecuciones de estos algoritmos, así como el número de ejecuciones que habría que

hacer en cada caso, partiendo siempre de una semilla distinta en cada ejecución para el generador de números aleatorios:

SH , SHE y SCH:

- Número de ejecuciones a realizar: 3.
- Número de hormigas: $m = 10$
- Criterio de parada): 1000 Iteraciones
- Valor de los parámetros de la regla de transición: $\alpha = 1$ y $\beta = 2$
- Número de hormigas elitistas para SHE será: $e = 5$.
- Valor del parámetro de evaporación en la actualización de feromona: $\rho = 0,1$.
- Cantidad inicial de feromona $\tau_0 = 1 / (n \cdot L)$, siendo n el tamaño de la instancia del problema y L el coste del circuito devuelto por la heurística *greedy* implementada.
- Aporte de feromona en el mecanismo de actualización:

$$f(C(S_k)) = \frac{1}{C(S_k)}$$

- Para SCH , Regla de actualización local (SCH): $\varphi=0.1$
- Regla de transición: $\alpha=1$, $\beta=2$, $q_0=0.98$ (SCH)

El alumno tendrá que contabilizar el número de evaluaciones (llamadas realizadas a la función de coste) producidas por los distintos algoritmos, que será empleado como una medida adicional de comparación de su calidad.

A partir de la experimentación efectuada, se obtendrán varias tablas en las que se incluirán los resultados obtenidos por cada algoritmo en cada instancia, así como la media y desviación típica de estos, con la forma de la Tabla 1.1.

Estudio experimental

Se construirá una tabla global de resultados con la estructura mostrada en la Tabla 1.1. En dicha tabla figura el mejor resultado de cada algoritmo y caso

	Caso 1				Caso 2				Caso 3			
Modelo	#Ev	F OBJ	Kms	Entr	#Ev	F OBJ	Kms	Entr	#Ev	F OBJ	Kms	Entr
SH												
SHE												
SCH												
Greedy A.												

Tabla 1.1. Resultados Globales

A partir de los datos mostrados en estas tablas, el alumno realizará un análisis de los resultados obtenidos, que influirá en la calificación de la práctica. En dicho análisis, se deben comparar, para cada instancia, los distintos algoritmos en términos de eficiencia, calidad de la solución, número de evaluaciones, mejor resultado individual obtenido y mejor resultado medio (robustez del algoritmo).

Fecha y Método de Entrega:

Antes del 30 de junio de 2026, no será necesaria defensa presencial o examen. Si fuese necesario alguna aclaración el profesor la solicitaría por correo electrónico.

Contenido: Código fuente y documentación con los resultados resumidos y las conclusiones del análisis respecto a las elecciones realizadas. Tenga en cuenta que el análisis que realice cuenta tanto o más como las implementaciones.

Rubrica:

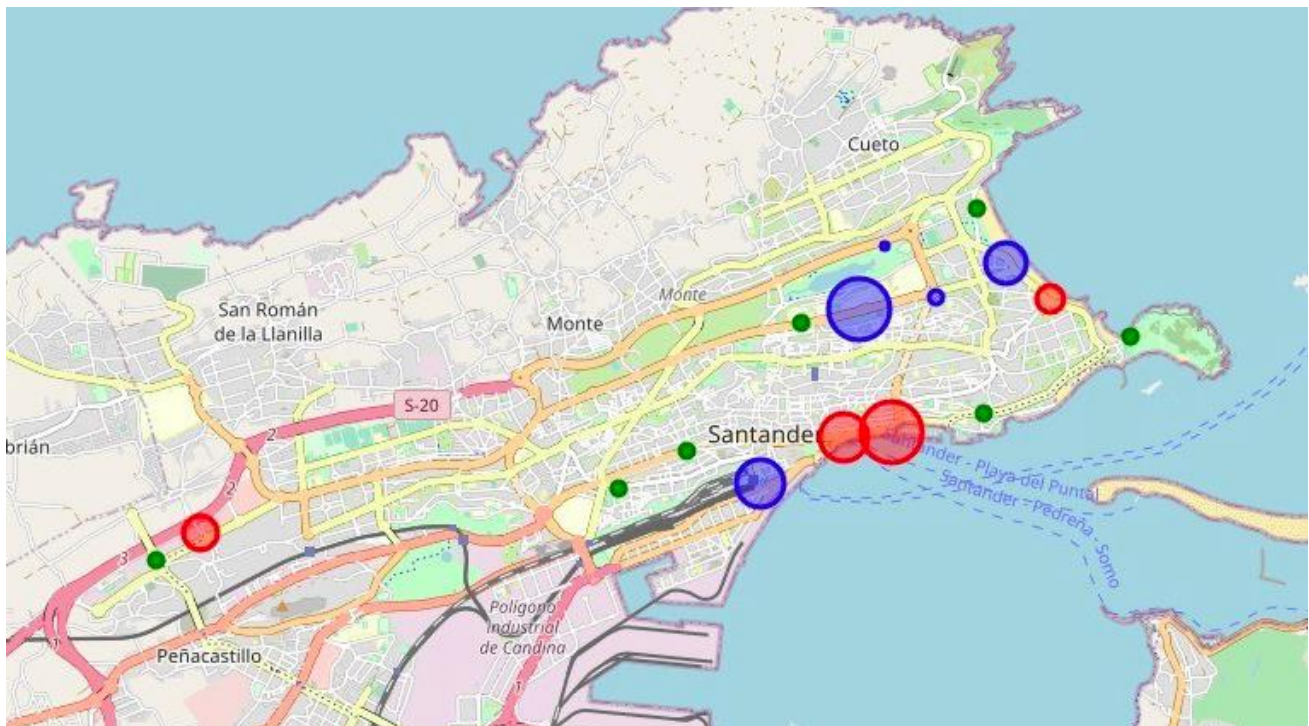
Aprobado (5-6-7): Algoritmos funcionales, tablas de SH y SHE SCH , Greedy aleatorizado

Notable-Sobresaliente (8-9-10): Realización de una pequeña animación GIF o Video consistente en el recorrido de la hormigas al principio , medio y fin de la optimización usando el mapa de Santander . Mostrar el rastro de feromona con líneas entre las estaciones.

Anexo: Como generar videos en base a Folium

- Generar varios mapas con Folium.
- Guardarlos como imágenes PNG.
- Unir esas imágenes en un GIF o vídeo usando Python.

Folium genera HTML interactivo, no imágenes directamente, así que normalmente se usa un navegador “headless” (como Chrome mediante Selenium) para capturar los mapas.



Ejemplo:

```
import folium
import os
import imageio.v2 as imageio
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
import time

# Carpeta de salida
os.makedirs("frames", exist_ok=True)

# Coordenadas simuladas
coords = [
    [37.3891, -5.9845],
    [37.3900, -5.9850],
    [37.3910, -5.9860],
    [37.3920, -5.9870],
]

# Configuración Chrome headless
options = Options()
options.add_argument("--headless")
options.add_argument("--window-size=800,600")

driver = webdriver.Chrome(options=options)

imagenes = []

for i, coord in enumerate(coords):

    # Crear mapa
    m = folium.Map(location=coord, zoom_start=15)

    folium.Marker(coord).add_to(m)

    html_file = f"frames/mapa_{i}.html"
    png_file = f"frames/mapa_{i}.png"

    # Guardar html
    m.save(html_file)

    # Abrir html y capturar imagen
    driver.get("file://" + os.path.abspath(html_file))

    time.sleep(1)

    driver.save_screenshot(png_file)

    imagenes.append(imageio.imread(png_file))

driver.quit()

# Crear GIF
imageio.mimsave("animacion.gif", imagenes, duration=0.8)

print("GIF generado")
```